

VLSI Testing and Testability Design

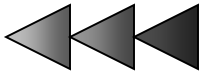
(超大型積體電路測試與可測試性設計)

Shyue-Kung Lu

Department of Electrical Engineering

National Taiwan University of Science and Technology

Objectives



- 學習VLSI Testing之基本原理與應用
- 運用VLSI Testing之原理於VLSI Design
- 介紹工業界廣泛使用之 DFT 軟體 (Boundary Scan, Memory BIST, ATPG, Scan-Based DFT)
- 了解系統晶片 (SOC) 與 SIP 測試之需求
- 掌握奈米技術下測試議題之趨勢 (3D-IC)
- 學習 AI Accelerator 測試原理

Syllabus

Text Book

L. T. Wang, C. W. Wu, and X. Q. Wen, "VLSI Test Principles and Architectures," Morgan Kaufmann Publishers, 2006.

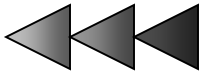
References

1. Michael L. Bushnell and Vishwani D. Agrawal, "Essentials of Electronic Testing," Kluwer Academic Publishers, 2000.
2. Miron Abramorici, et. al., "Digital Systems Testing and Testability Design," Computer Science Press, 1995.
3. Parag K. Lala, "Digital Circuit Testing and Testability," Academic Press, 1997.
4. Papers

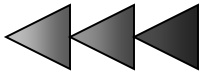
Grading

1. 期中考 30% 2. 期末考 40% 3. 作業 30%

Contents



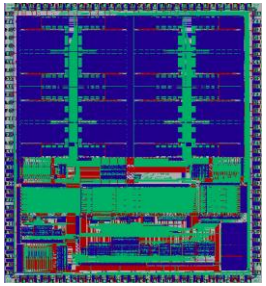
- Introduction
- Design for testability
- Logic and fault simulation, Automatic test pattern generation (ATPG)
- Logic built-in self-test, Test compression
- Memory testing and built-in self-test
- Memory diagnosis and built-in self-repair
- On-Line Memory Test and Repair
- Boundary scan (IEEE 1149.1) and Board-Level Testing
- IEEE 1500 and Core-based SOC Testing
- Test and repair techniques for emerging memory technologies (Flash memory, PCM, NROM)
- 3D IC test and repair
- Test techniques for AI accelerators



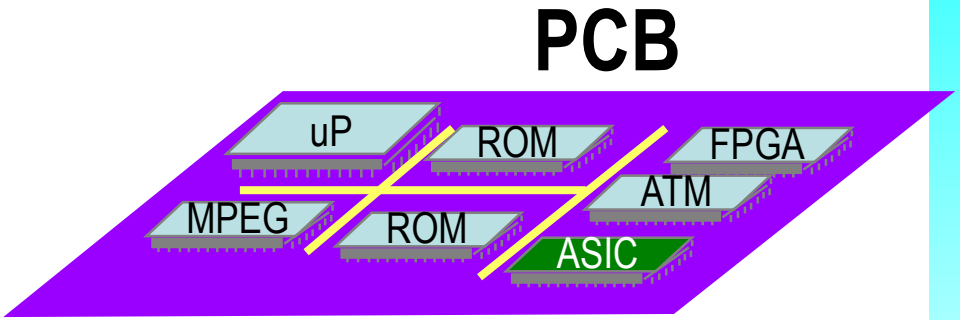
Chapter 1

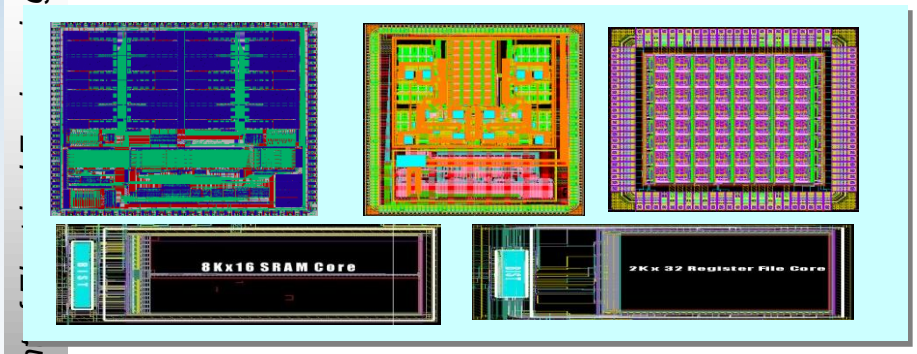
Introduction

Maintaining Quality & Reliability

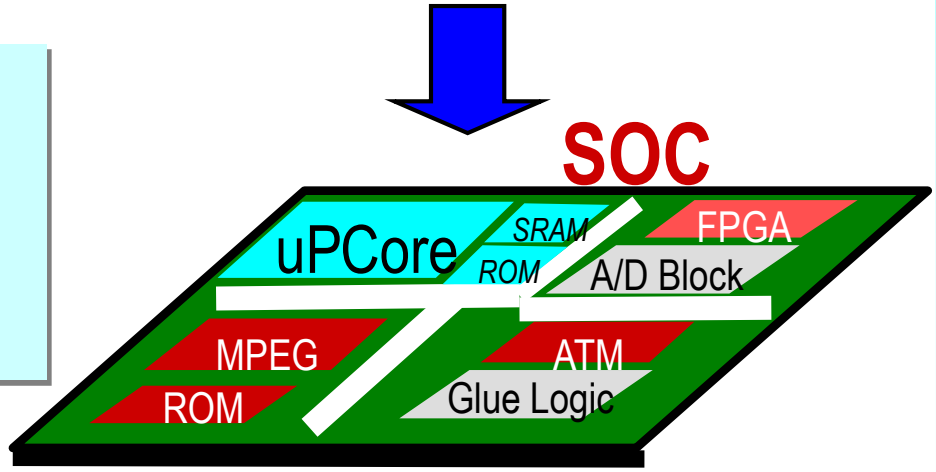


DL=200





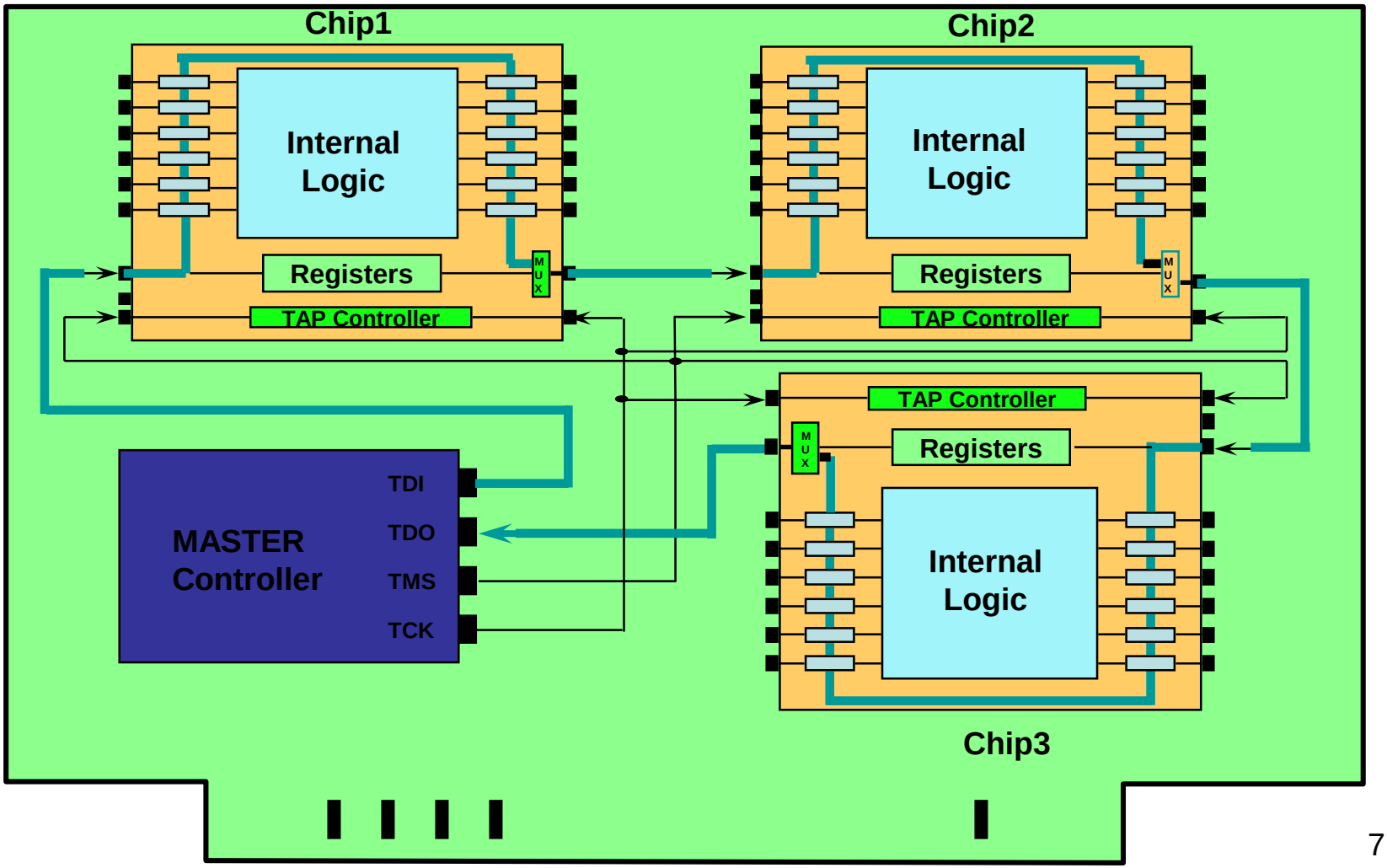
DL=?



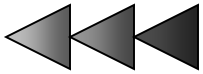
Virtual Component Notion

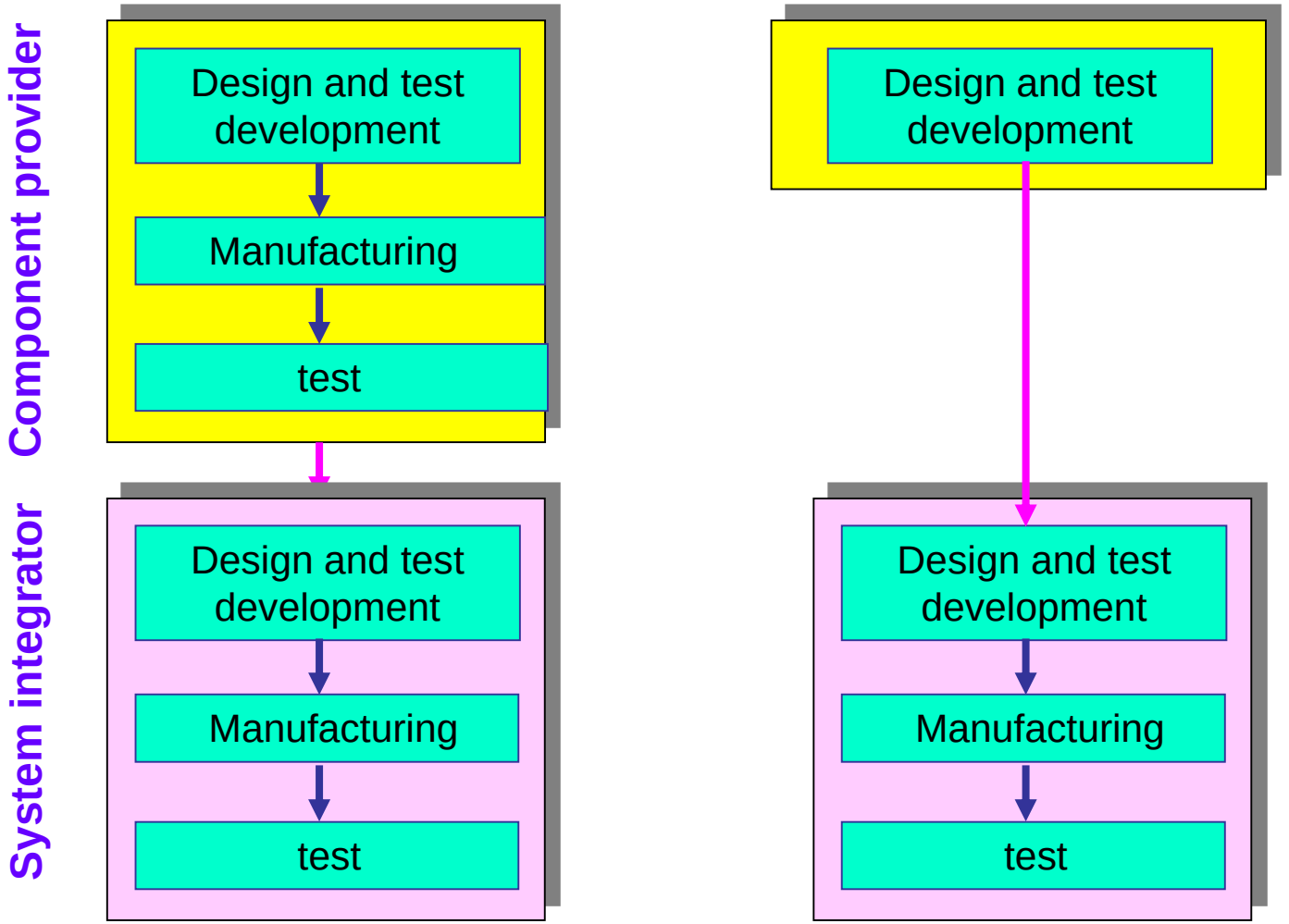
A Printed Circuit Board With 1149.1

(Ring configuration, test controller on board)

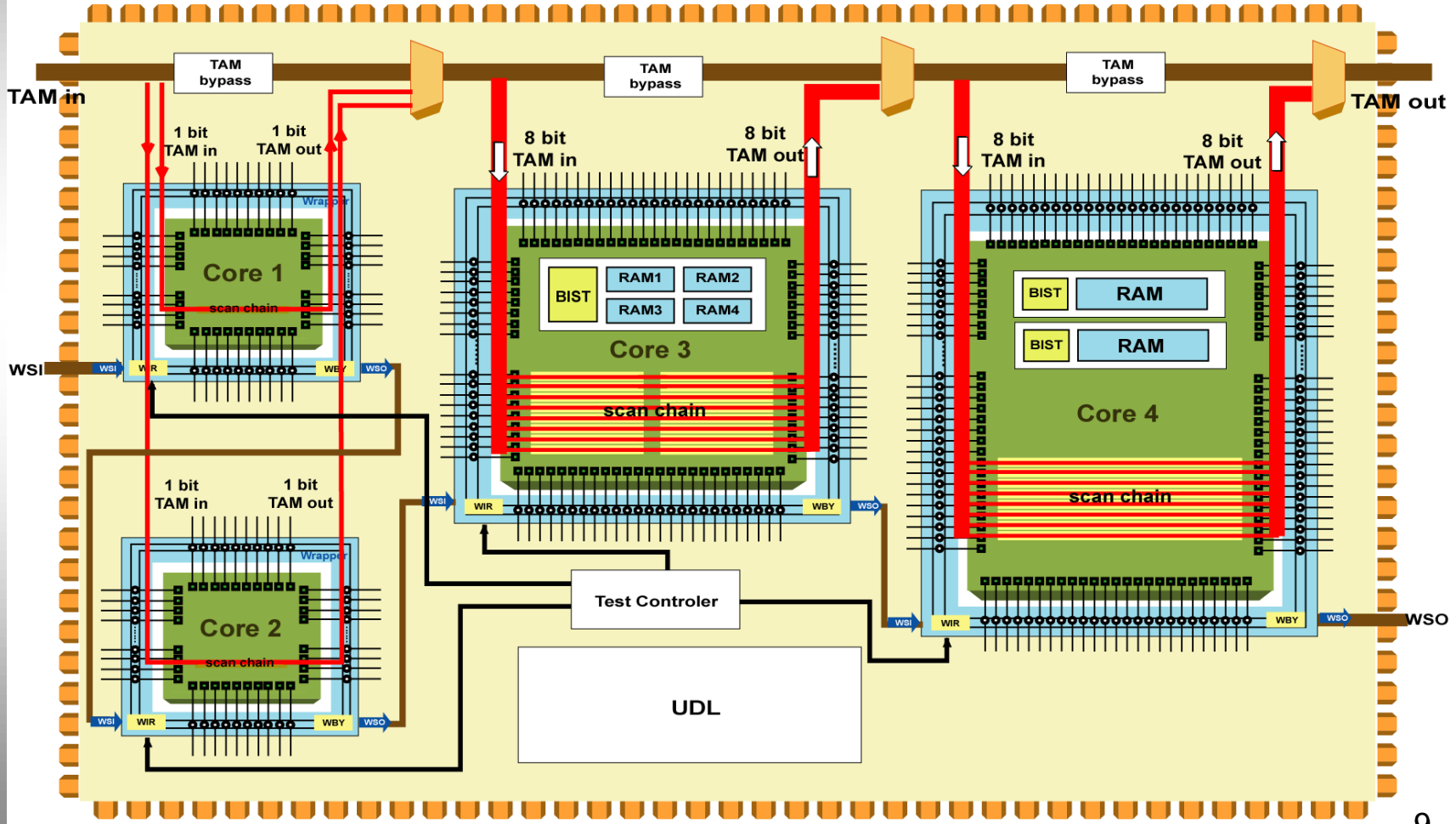


SOC Test Specifics



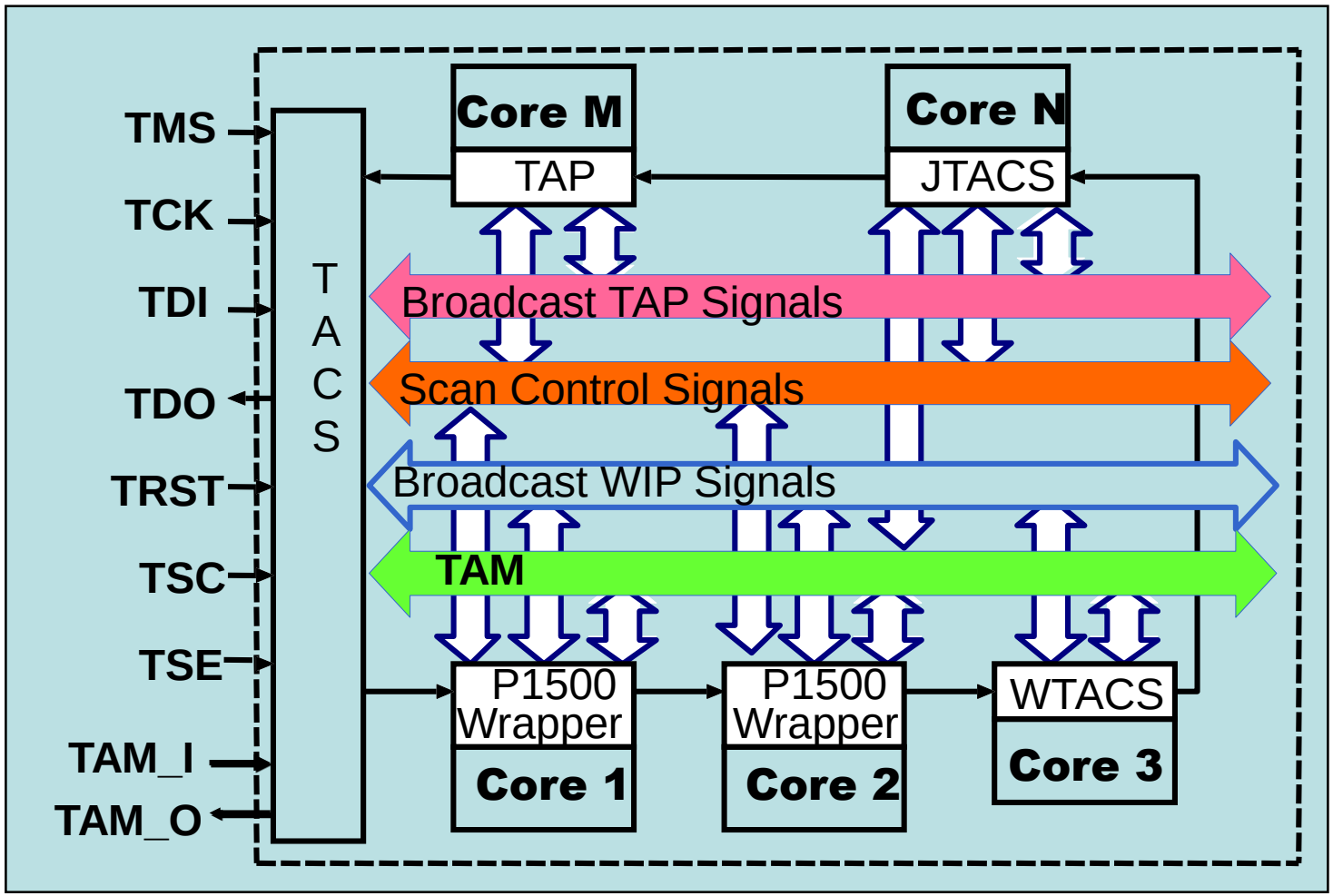


Implementation Example



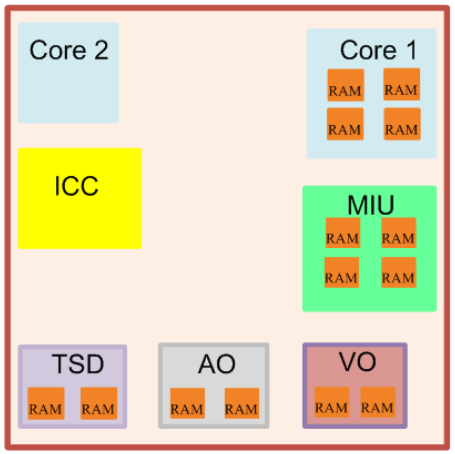
TACS-based SOC Architecture

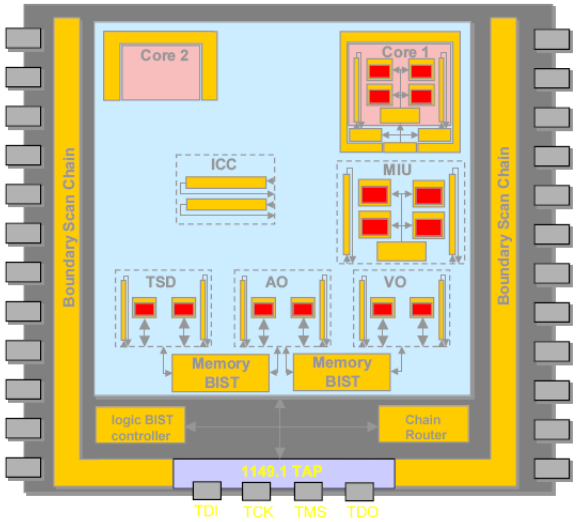
- Support both P1500 and IEEE 1149.1 wrapped cores



An Example of Core-based SOC Testing

- Total 250K gates of digital logic
  - 29 embedded memories
  - 2 distinct cores and 5 additional
- Core 1: 100K gates, 10 embedded memories
- Core 2: Legacy Core with functional test
- ICC : 10K gates
- TSD: 20K gates, 3 embedded memories
- MIU: 40K gates, 10 embedded memories
- VO: 20K gates, 3 embedded memories
- AO: 20K gates, 3 embedded memories

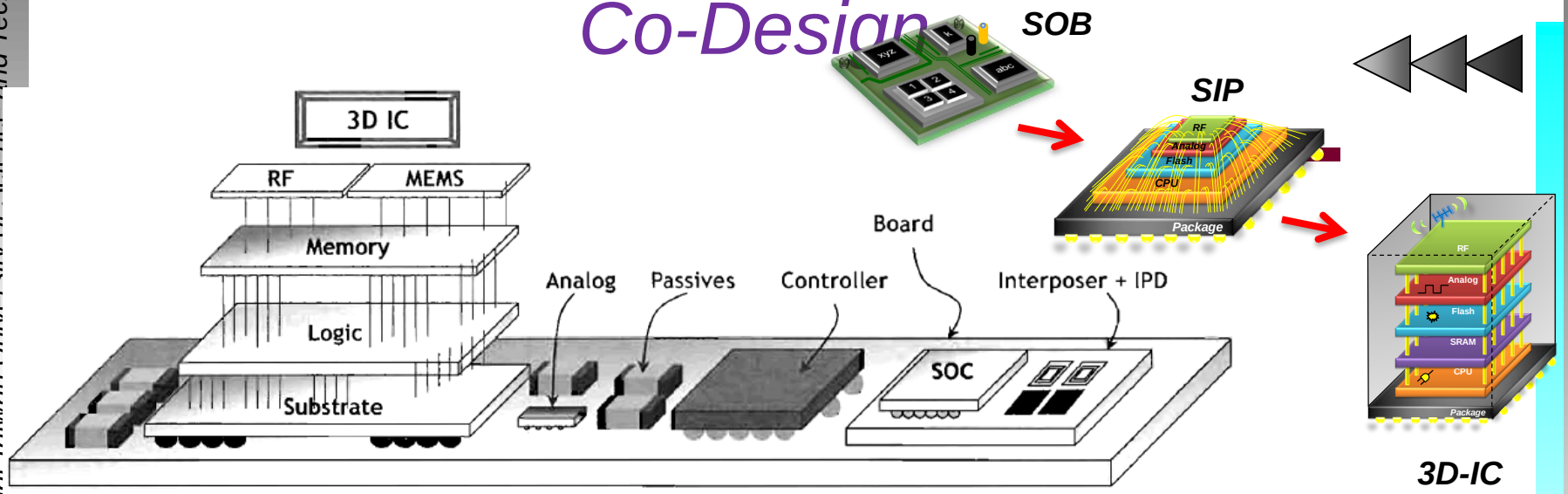




Pre-Layout Testability Costs

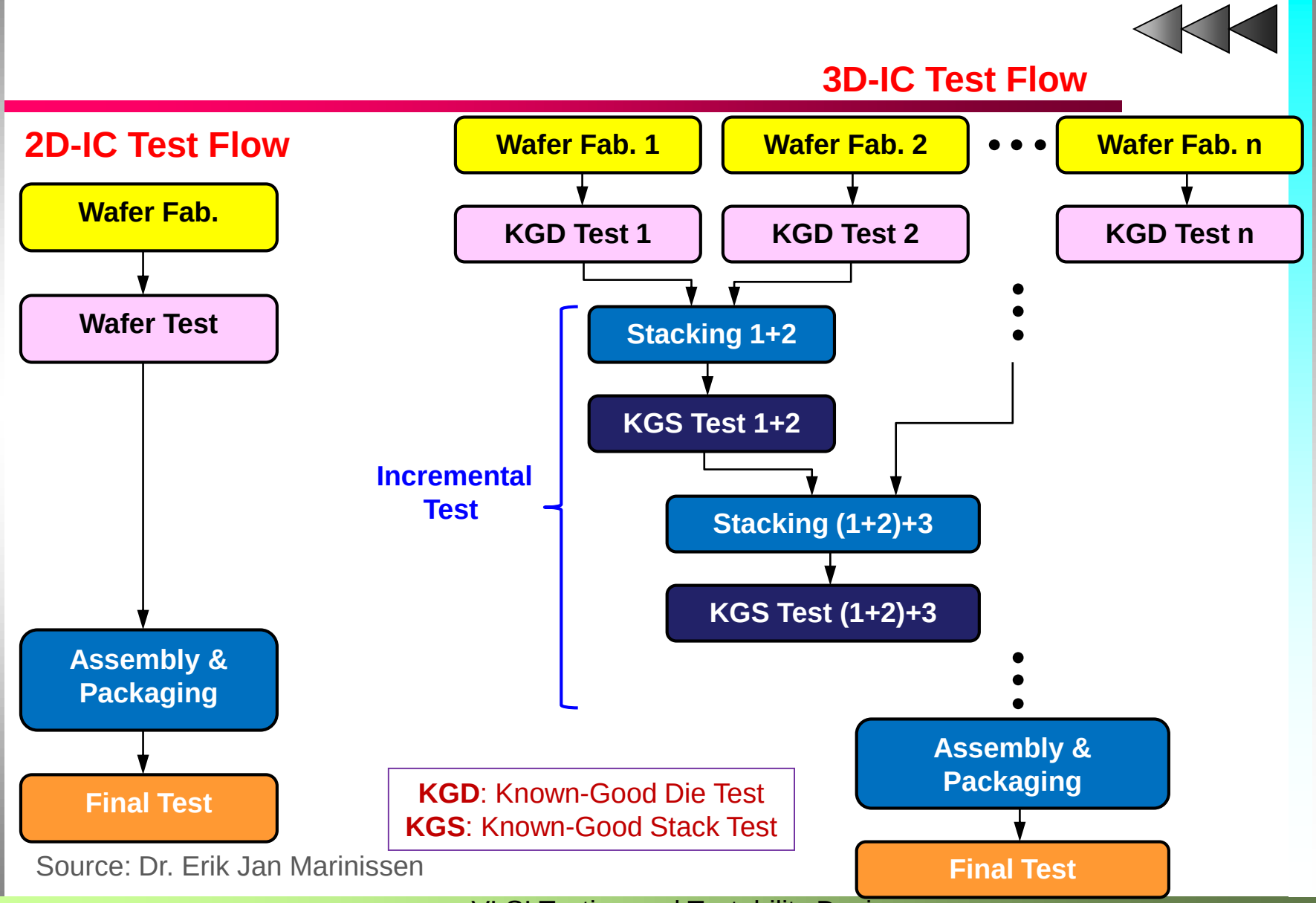
Block	Core 1: VD	Core 2: GPIO	JSAT3-Core	JSAT3-Chip	All Cores & Chip Level
memBIST Controller	0.84%		1.58%	0%	2.43%
logicBIST Controllers	0.74%		0%	2.18%	2.91%
logicBIST Routers	0.13%		0%	1.02%	1.14%
Total Area for BIST	1.7%	0%	1.5%	3.19%	6.48%
Full Scan	2.78%		3.48%	0%	6.26%
Memory Collars	1.72%	0%	4.19%	0%	5.91%
Socket Elements	0.90%	0.32%	0%	0%	1.22%
BSC & TAP Controller	0%	0%	0%	1.58%	1.58%
Total Infrastructure	5.39%	0.32%	7.67%	1.58%	14.96%
Total DFT Area	7.09%	0.32%	9.25%	4.77%	21.44%
Fault Coverage with logicBIST	99.09%				97.80%
Fault Coverage with logicBIST and Deterministic Vectors					99.20%

Current Research Topics—*3D IC Co-Design*



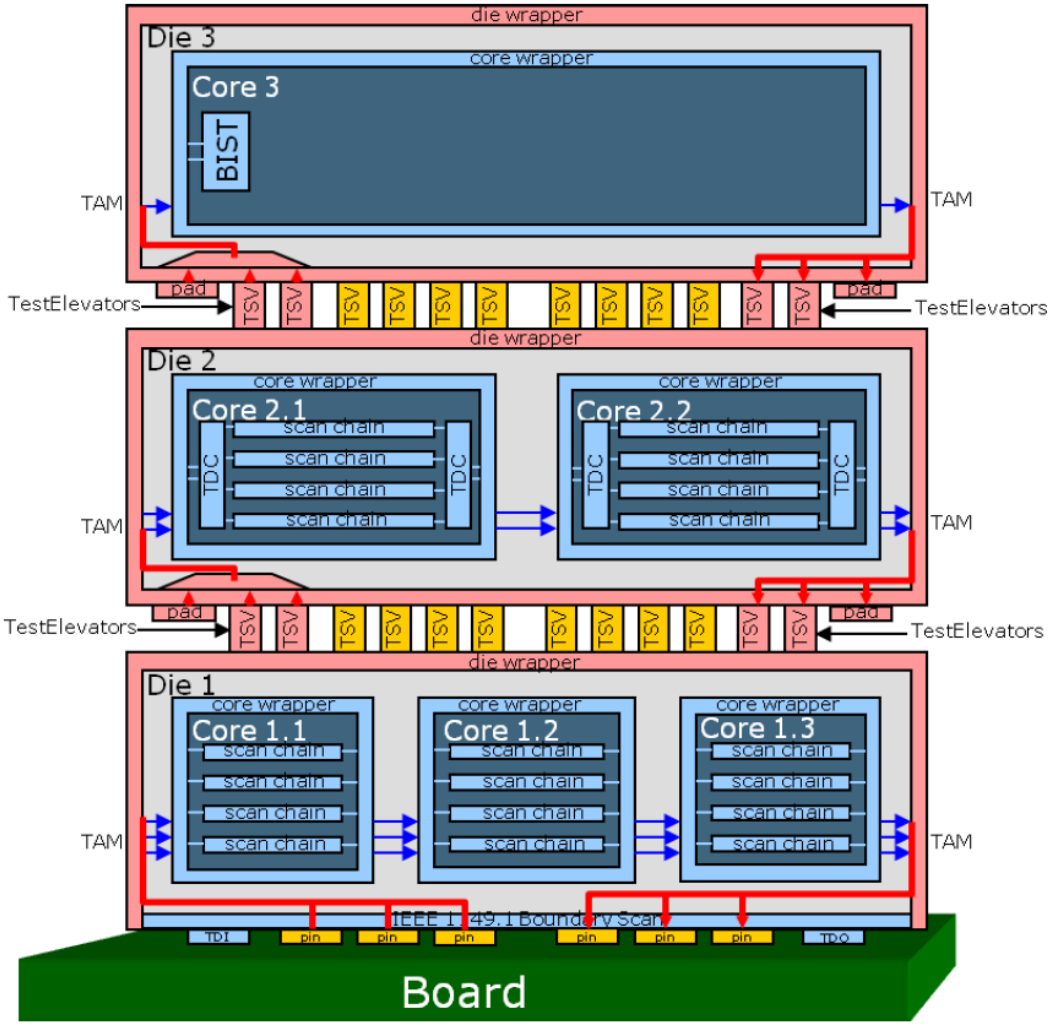
	Challenges	Gap
Design	3D IC EDA Tool Environment 3D IC Design Flow from IC to System	No Proven Product
Electrical	TSV Characterization (Electrical + Stress) System-Level SI/PI Validation & Sign-Off Flow	Validation Flow & Tool
Thermal	3D IC Thermal Models Thermal-Aware Design & Management	Support Flow & Tool
Mechanical	System-Level Package Reliability	No Test Standard
Applications	CMOS Image Sensor, Video Coding, etc.	More Applications

Challenges—*Testing*

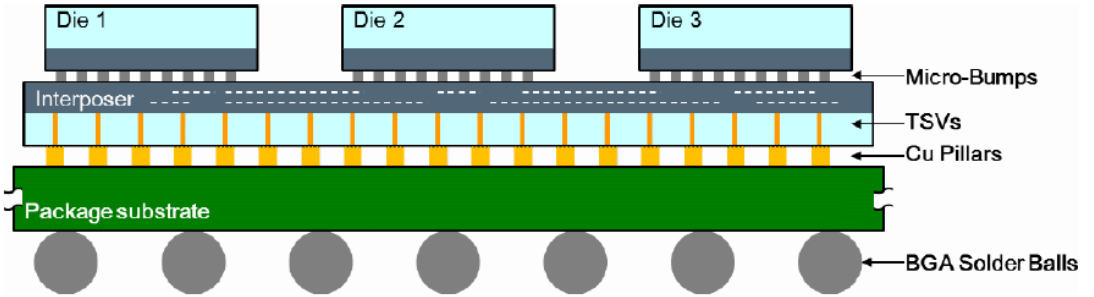


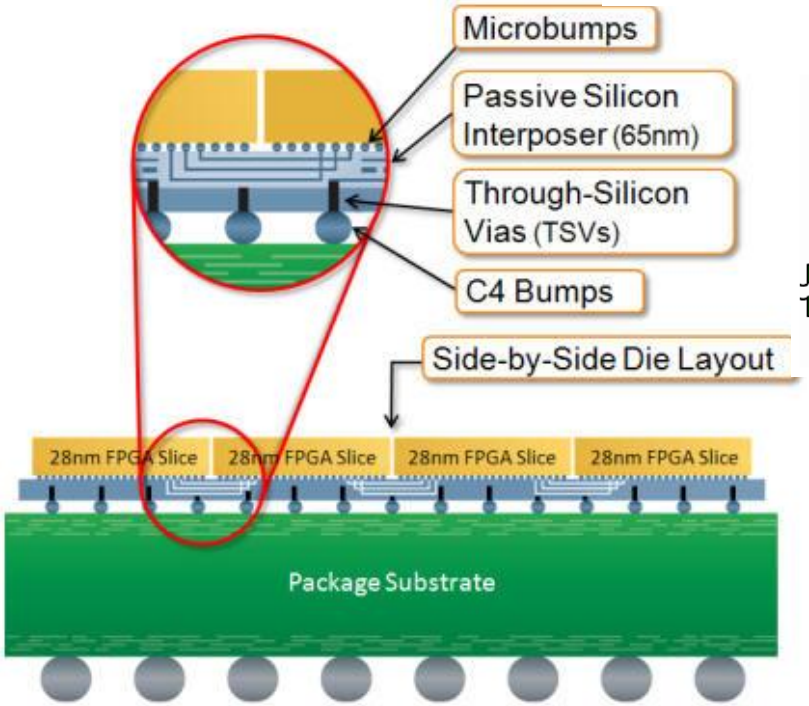
Source: Dr. Erik Jan Marinissen

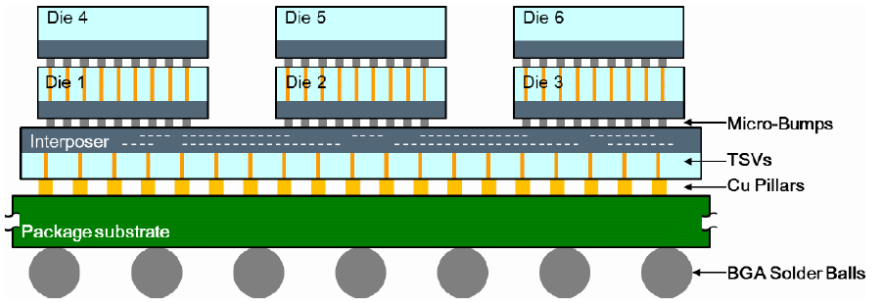
DFT Architecture for 3D IC



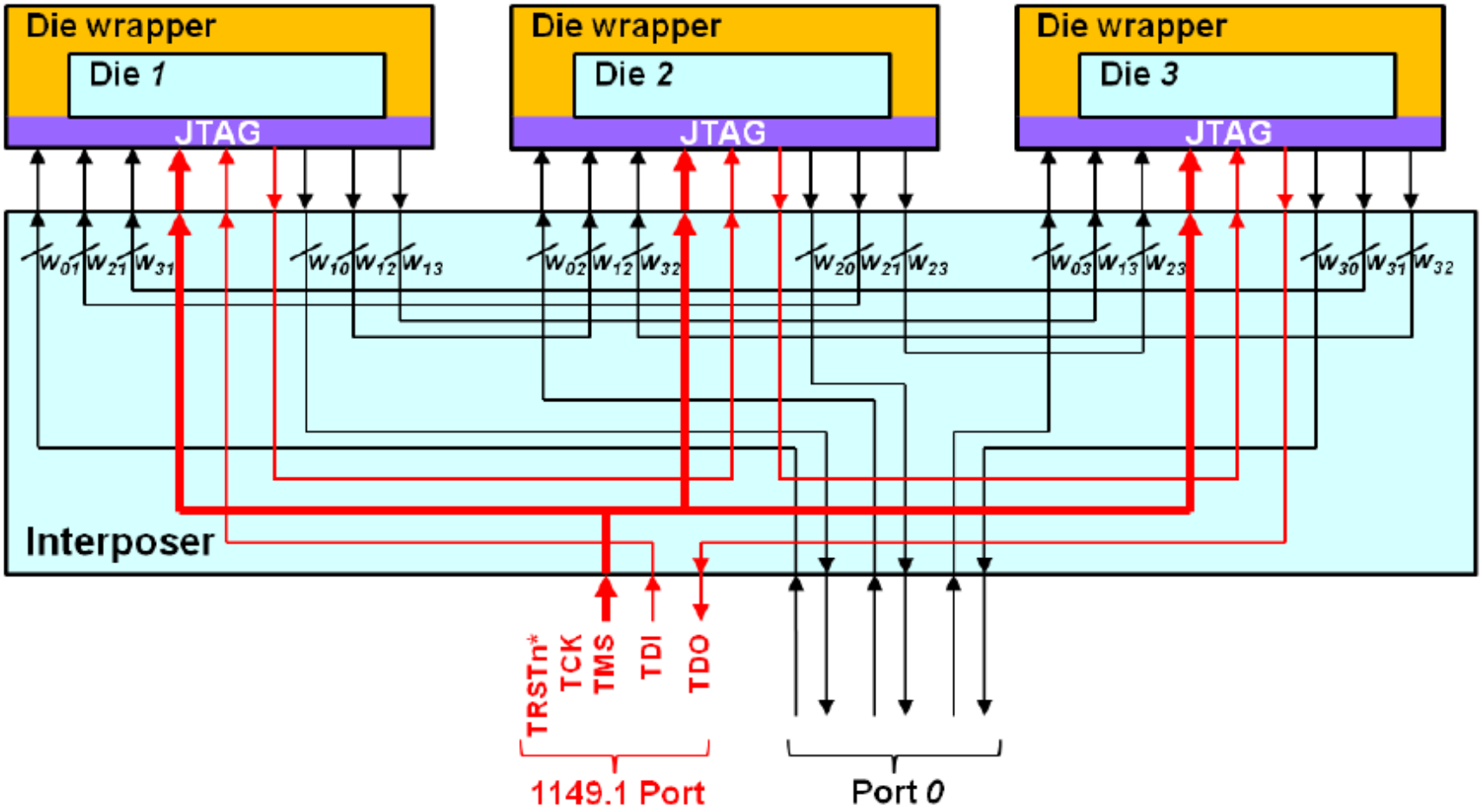
2.5D IC


Die 1
Die 2
Die 3
Micro-Bumps
Interposer
TSVs
Cu Pillars
Package substrate
BGA Solder Balls


Microbumps
Passive Silicon Interposer (65nm)
Through-Silicon Vias (TSVs)
C4 Bumps
Side-by-Side Die Layout
28nm FPGA Slice
28nm FPGA Slice
28nm FPGA Slice
28nm FPGA Slice
Package Substrate


Die 4
Die 5
Die 6
Die 1
Die 2
Die 3
Micro-Bumps
Interposer
TSVs
Cu Pillars
Package substrate
BGA Solder Balls

DFT Architecture for 2.5D IC

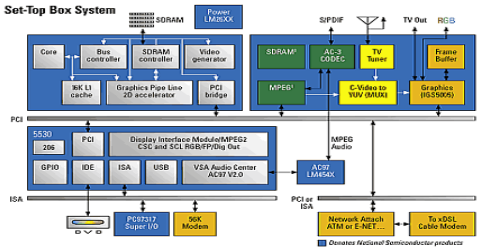


Chip Design & Manufacturing Flow



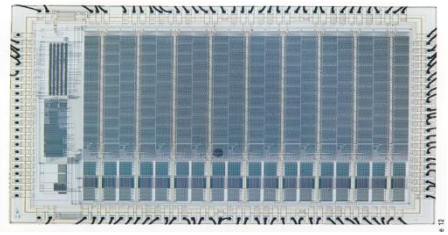
Idea

Architecture Design



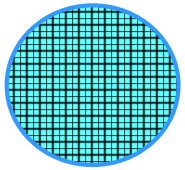
Block diagram

Circuit & Layout Design



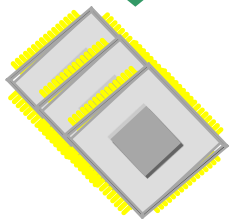
Layout

IC Fabrication



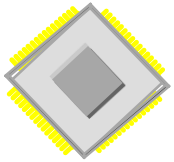
Wafer (hundreds of dies)

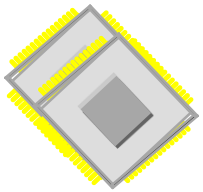
Sawing & Packaging



Final chips

Testing





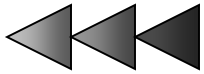
Bad chips

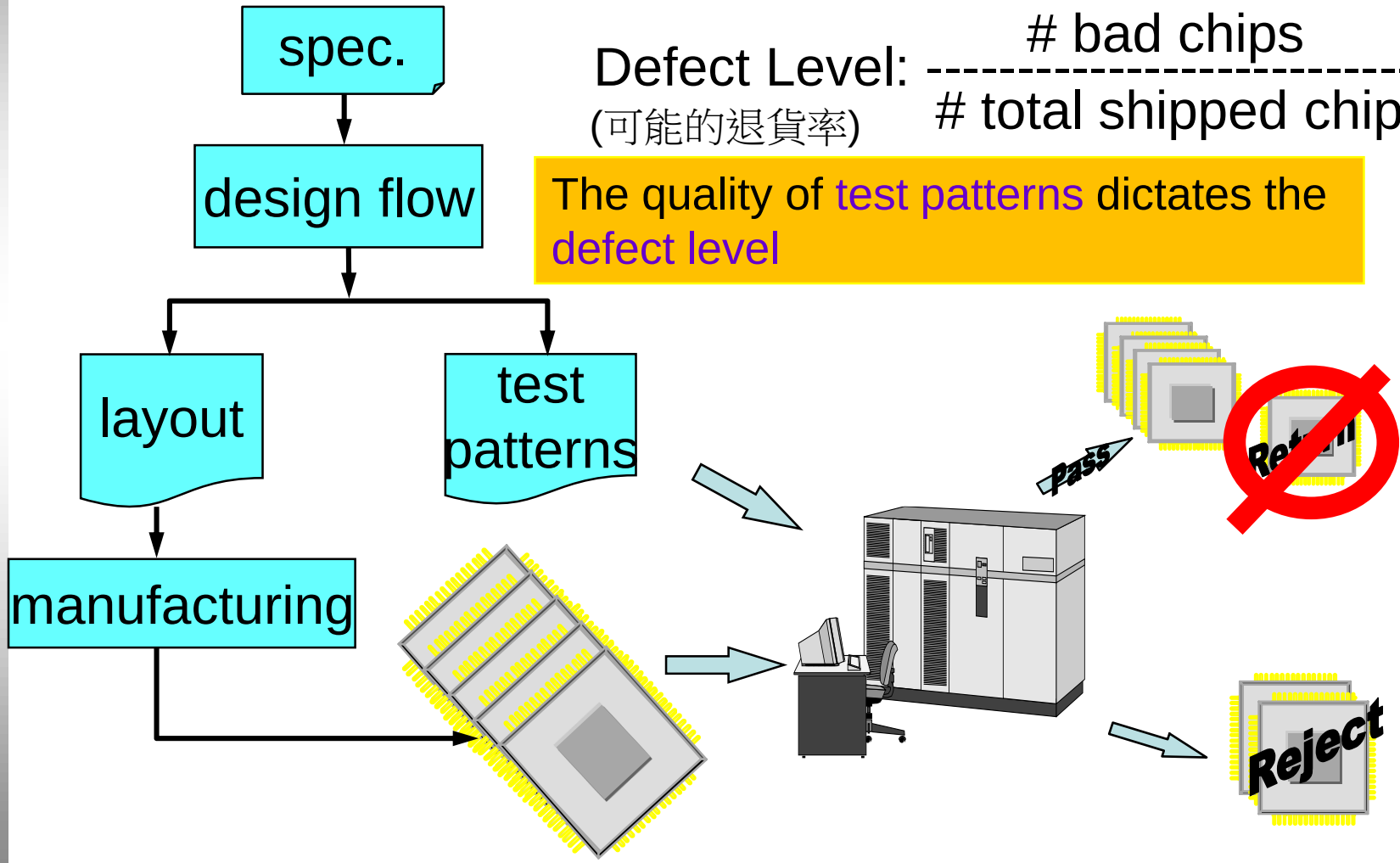
Good chips



customers

Design Flow and Defect Level



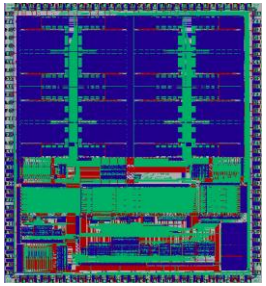


Defect Level: $\frac{\text{\# bad chips}}{\text{\# total shipped chips}}$

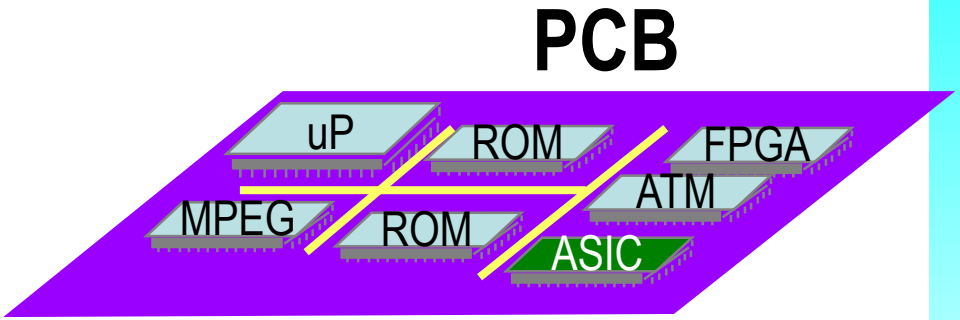
(可能的退貨率)

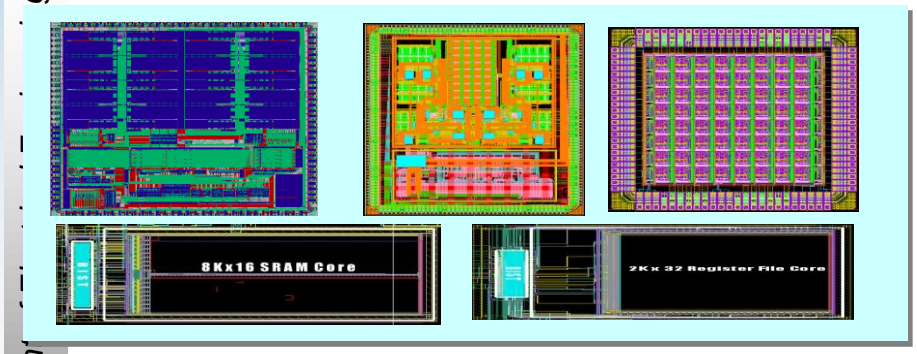
The quality of test patterns dictates the defect level

Maintaining Quality & Reliability

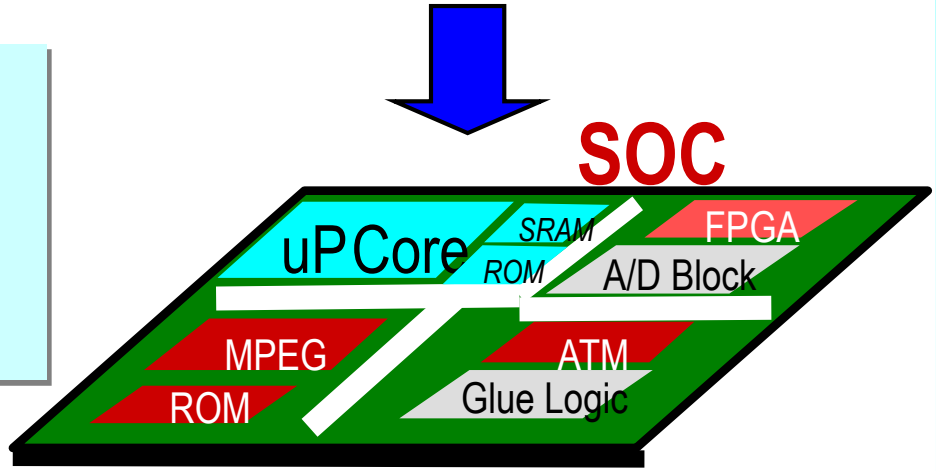


DL=200

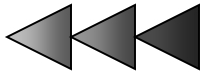


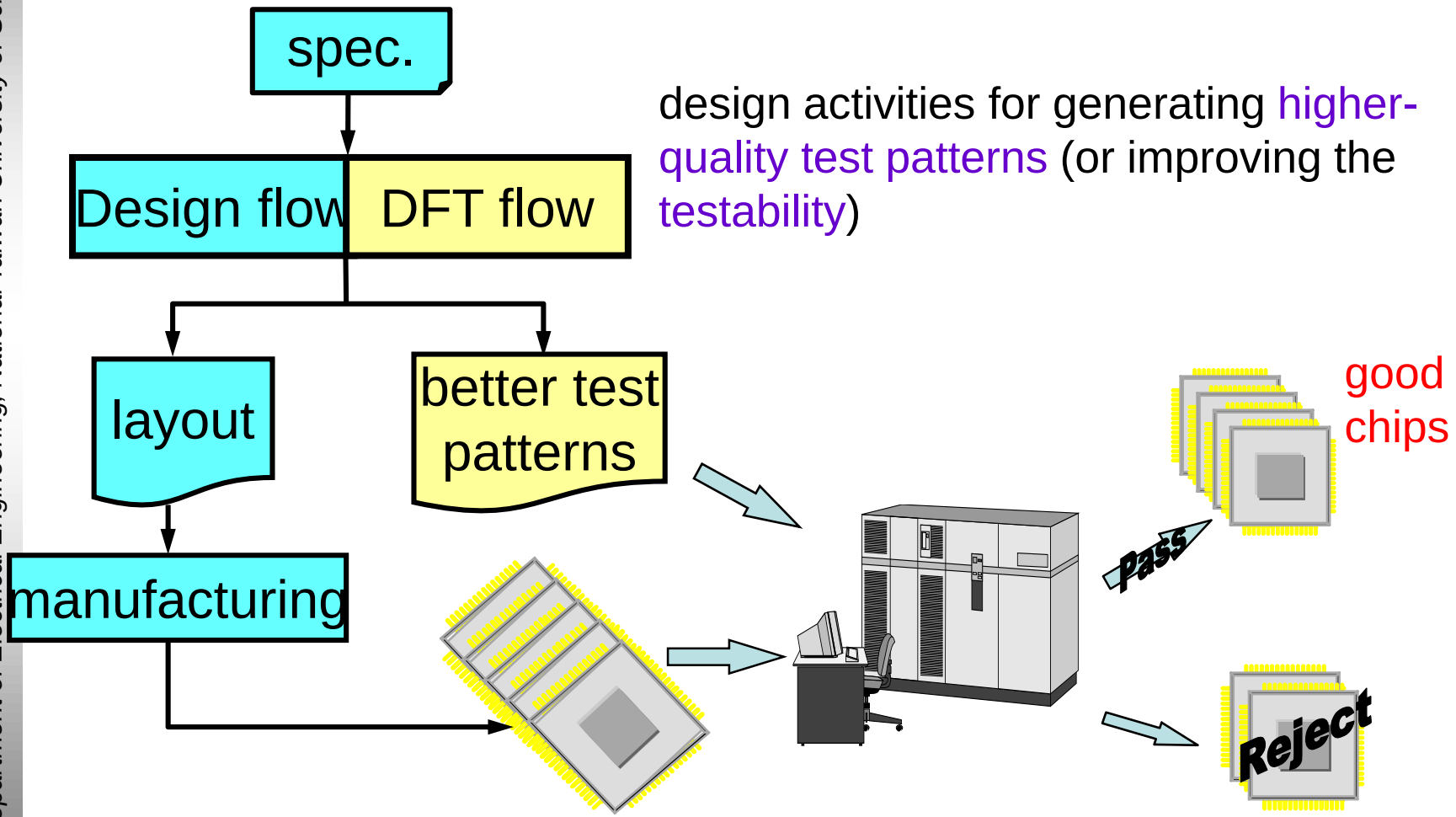


DL=?



Design for Testability (DFT)

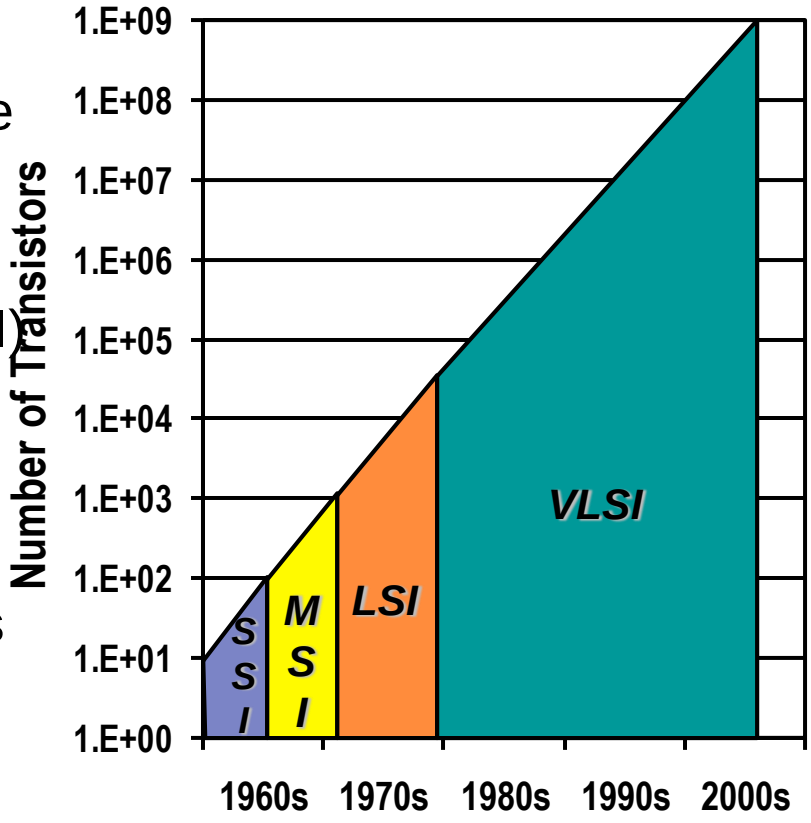




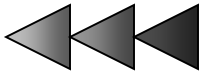
design activities for generating higher-quality test patterns (or improving the testability)

Introduction

- Integrated Circuits (ICs) have grown in size and complexity since the late 1950's
  - Small Scale Integration (SSI)
  - Medium Scale Integration (MSI)
  - Large Scale Integration (LSI)
  - Very Large Scale Integration (VLSI)
- *Moore's Law*: scale of ICs doubles every 18 months
  - Growing size and complexity poses many and new testing challenges



Importance of Testing



- Moore's Law results from decreasing feature size (dimensions)
  - from 10s of μm to 10s of nm for transistors and interconnecting wires
- Operating frequencies have increased from 100KHz to several GHz
- Decreasing feature size increases probability of defects during manufacturing process
  - A single faulty transistor or wire results in faulty IC
  - Testing required to guarantee fault-free products

Importance of Testing

- *Rule of Ten*: cost to detect faulty IC increases by an order of magnitude as we move from:
  - device → PCB → system → field operation
    - Testing performed at all of these levels
- Testing also used during
  - Manufacturing to improve yield
    - Failure mode analysis (FMA)
  - Field operation to ensure fault-free system operation
    - Initiate repairs when faults are detected



Defect detected during IC testing



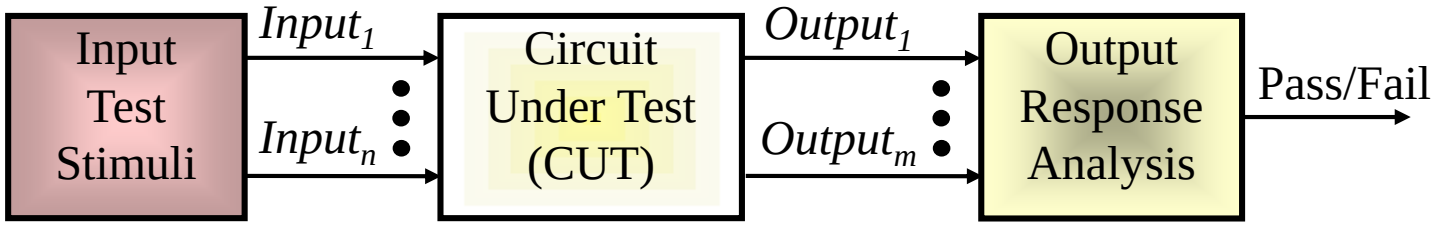
Defect detected during system testing



Defect detected during field testing

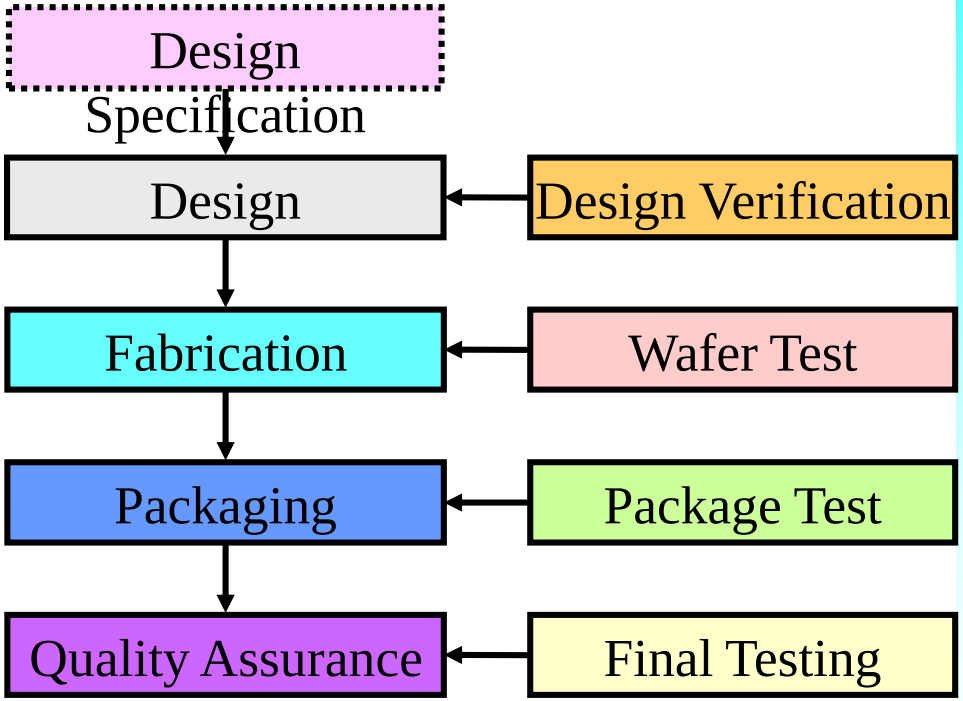
Testing During VLSI Life Cycle

- Testing typically consists of
  - Applying set of test stimuli to
  - Inputs of *circuit under test* (CUT), and
  - Analyzing output responses
    - If incorrect (fail), CUT assumed to be faulty
    - If correct (pass), CUT assumed to be fault-free



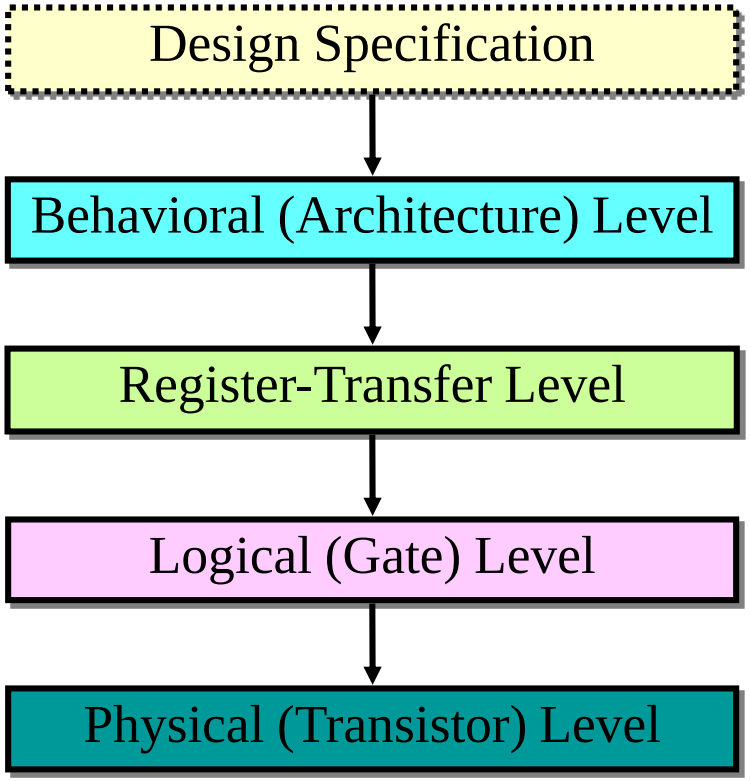
Testing During VLSI Development

- Design verification targets design errors
  - Corrections made prior to fabrication
- Remaining tests target manufacturing defects
  - A defect is a flaw or physical imperfection that can lead to a fault

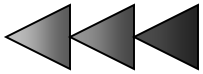


Design Verification

- Different levels of abstraction during design
  - CAD tools used to synthesize design from RTL to physical level
- Simulation used at various level to test for
  - Design errors in behavioral or RTL
  - Design meeting system timing requirements after synthesis



Yield and Reject Rate



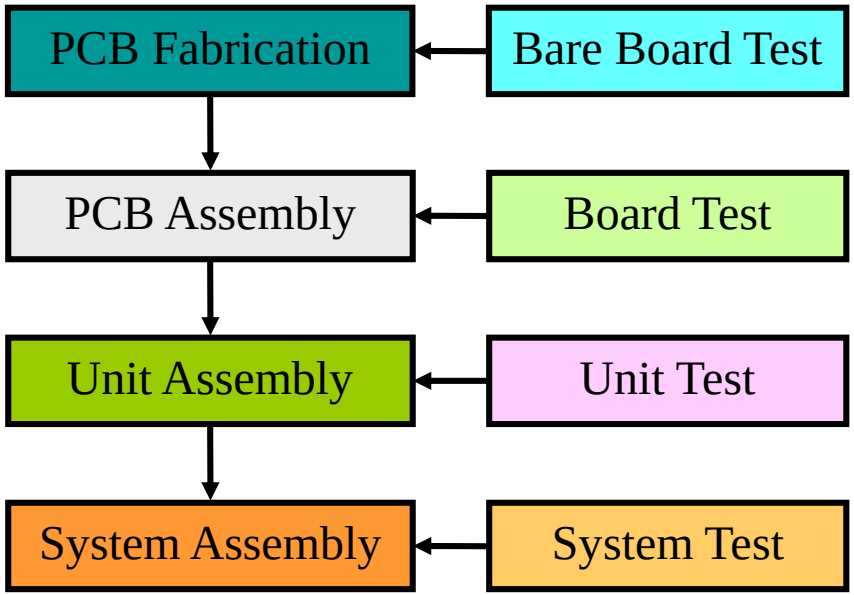
- We expect faulty chips due to manufacturing defects
  - Called yield
- 2 types of yield loss
  - Catastrophic–due to random defects
  - Parametric–due to process variations
- Undesirable results during testing
  - Faulty chip appears to be good (passes test)
    - Called reject rate
  - Good chip appears to be faulty (fails test)
    - Due to poorly designed tests or lack of DFT

$$yield = \frac{number\ of\ acceptable\ parts}{total\ number\ of\ parts\ fabricated}$$

$$reject\ rate = \frac{number\ of\ faulty\ parts\ passing\ final\ test}{total\ number\ of\ parts\ passing\ final\ test}$$

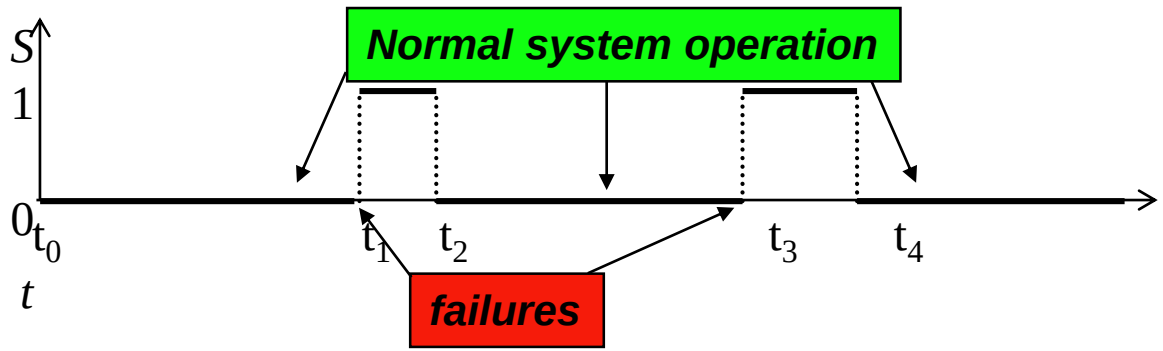
Electronic System Manufacturing

- A system consists of
  - PCBs that consist of
    - VLSI devices
- PCB fabrication similar to VLSI fabrication
  - Susceptible to defects
- Assembly steps also susceptible to defects
  - Testing performed at all stages of manufacturing



System-Level Operation

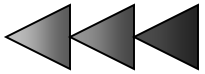
- Exponential failure law
  - Interval of normal system operation is random number exponentially distributed
- Reliability
  - Probability that system will operate normally until time *t*
  - Failure rate, λ, is sum of individual component failure rates, λ_i



$$P(T_n > t) = e^{-\lambda t}$$

$$\lambda = \sum_{i=0}^{k} \lambda_i$$

System-Level Operation



- Mean Time Between Failures (MTBF)
- Repair time (R) also assumed to obey exponential distribution
  - μ is repair rate
- Mean Time To Repair (MTTR)
- Fraction of time that system is operating normally called system availability
  - High reliability systems have system availabilities greater than 0.9999
    - Referred to as “four 9s”

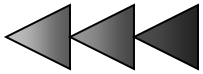
$$MTBF = \int_0^\infty e^{-\lambda t} dt = \frac{1}{\lambda}$$

$$P(R > t) = e^{-\mu t}$$

$$MTTR = \frac{1}{\mu}$$

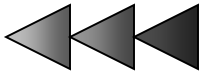
$$system\ availability = \frac{MTBF}{MTBF + MTTR}$$

System-Level Testing



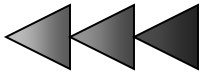
- Testing required to ensure system availability
- Types of system-level testing
  - On-line testing – concurrent with system operation
  - Off-line testing – while system (or portion of) is taken out of service
    - Performed periodically during low-demand periods
    - Used for diagnosis (identification and location) of faulty replaceable components to improve repair time

Test Generation



- A test is a sequence of test patterns, called test vectors, applied to the CUT whose outputs are monitored and analyzed for the correct response
  - Exhaustive testing–applying all possible test patterns to CUT
  - Functional testing–testing every truth table entry for a combinational logic CUT
    - Neither of these are practical for large CUTs
- Fault coverage is a quantitative measure of quality of a set of test vectors

Test Generation



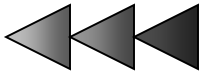
- Fault coverage for a given set of test vectors
- 100% fault coverage may be impossible due to undetectable faults

$$fault\ coverage = \frac{number\ of\ detected\ faults}{total\ number\ of\ faults}$$

- *Reject rate* = 1 – *yield*$^{(1 - fault\ coverage)}$
  - A PCB with 40 chips, each with 90% fault coverage and 90% yield, has a reject rate of 41.9%
    - Or 419,000 defective parts per million (PPM)

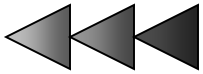
$$fault\ detection\ efficiency = \frac{number\ of\ detected\ faults}{total\ number\ of\ faults - number\ of\ undetectable\ faults}$$

Test Generation



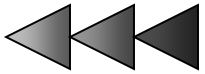
- *Goal*: find efficient set of test vectors with maximum fault coverage
- Fault simulation used to determine fault coverage
  - Requires fault models to emulate behavior of defects
- A good fault model:
  - Is computationally efficient for simulation
  - Accurately reflects behavior of defects
- No single fault model works for all possible defects

Fault Models



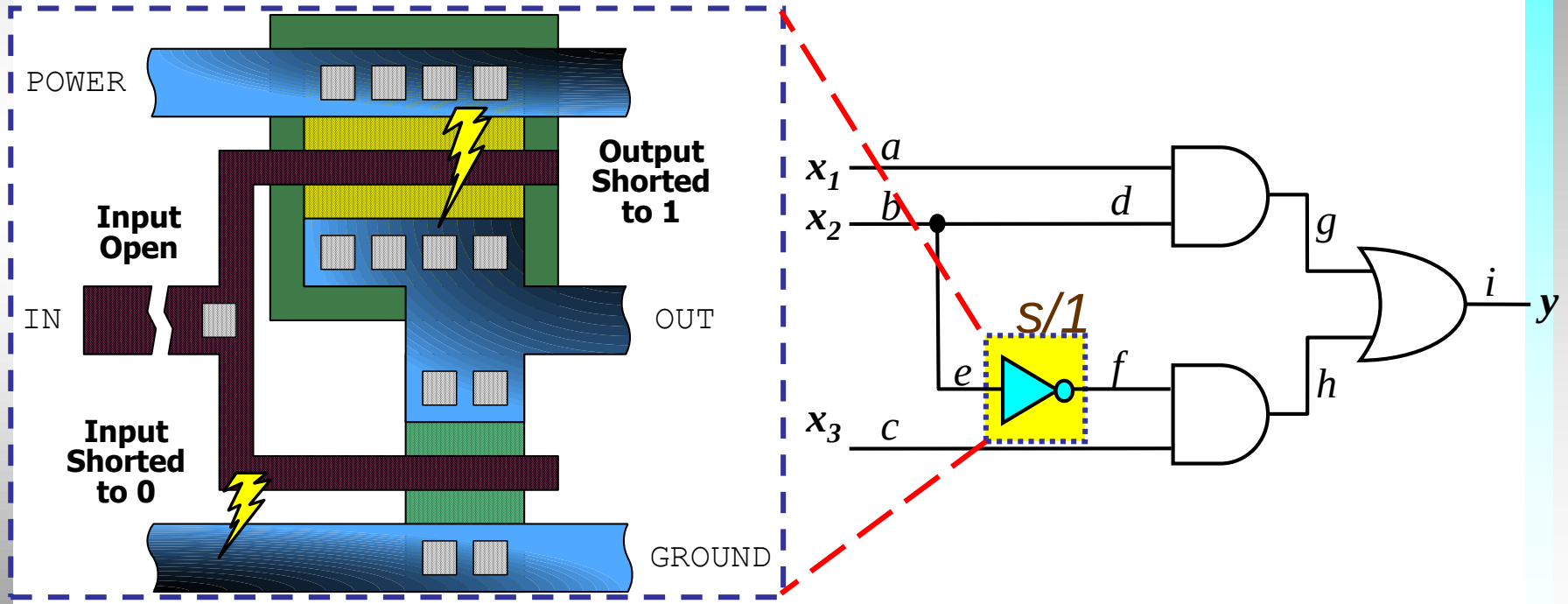
- A given fault model has *k* types of faults
  - *k* = 2 for most fault models
- A given circuit has *n* possible fault sites
- Multiple fault model–circuit can have multiple faults (including single faults
  - Number of multiple fault = $(k+1)^n-1$
    - Each fault site can have 1-of-*k* fault types or be fault-free
    - The "-1" represents the fault-free circuit
  - Impractical for anything but very small circuits
- Single fault model – circuit has only 1 fault
  - Number of single faults = $k \times n$
  - Good single fault coverage generally implies good multiple fault coverage

Fault Models



- Equivalent faults
  - One or more single faults that have identical behavior for all possible input patterns
  - Only one fault from a set of equivalent faults needs to be simulated
- Fault collapsing
  - Removing equivalent faults
    - Except for one to be simulated
  - Reduces total number of faults
    - Reduces fault simulation time
    - Reduces test pattern generation time

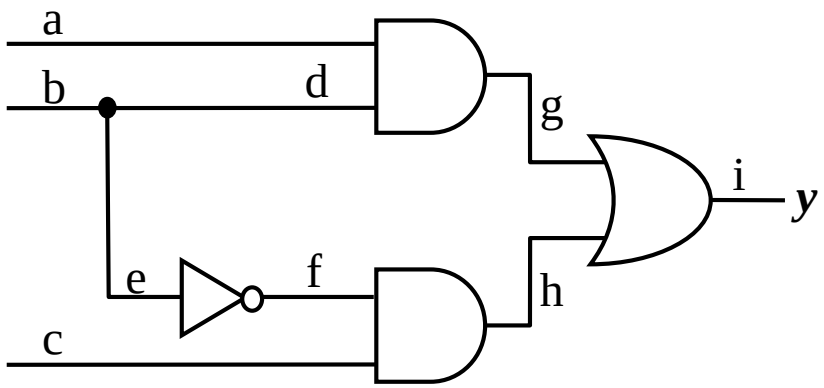
Stuck-at Fault



Stuck-at Faults

- Any line can be
  - Stuck-at-0 (SA0)
  - Stuck-at-1 (SA1)

  # fault types: k=2
- Example circuit:
  - # fault sites: n=9
  - # single faults =2 × 9=18



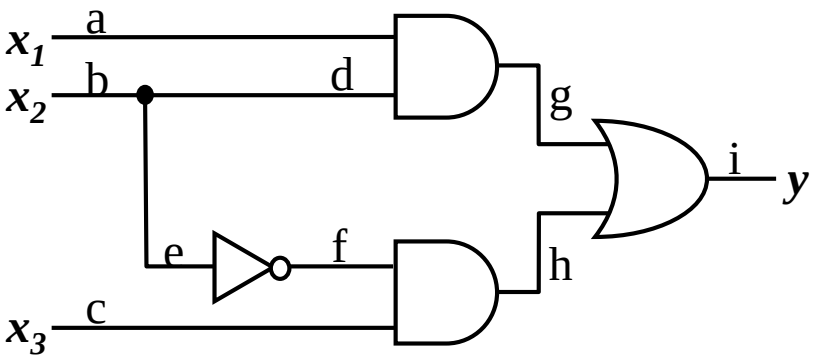
$x_1x_2x_3$	000	001	010	011	100	101	110	111
y	0	1	0	0	0	1	1	1
a SA0	0	1	0	0	0	1	0	0
a SA1	0	1	1	1	0	1	1	1
b SA0	0	1	0	1	0	1	0	1
b SA1	0	0	0	0	1	1	1	1
c SA0	0	0	0	0	0	0	1	1
c SA1	1	1	0	0	1	1	1	1
d SA0	0	1	0	0	0	1	0	0
d SA1	0	1	0	0	1	1	1	1
e SA0	0	1	0	1	0	1	1	1
e SA1	0	0	0	0	0	0	1	1
f SA0	0	0	0	0	0	0	1	1
f SA1	0	1	0	1	0	1	1	1
g SA0	0	1	0	0	0	1	0	0
g SA1	1	1	1	1	1	1	1	1
h SA0	0	0	0	0	0	0	1	1
h SA1	1	1	1	1	1	1	1	1
i SA0	0	0	0	0	0	0	0	0
i SA1	1	1	1	1	1	1	1	1

Stuck-at Faults

- Valid test vectors
  - Faulty circuit differs from good circuit
- Necessary vectors:

  011 detects f SA1, e SA0

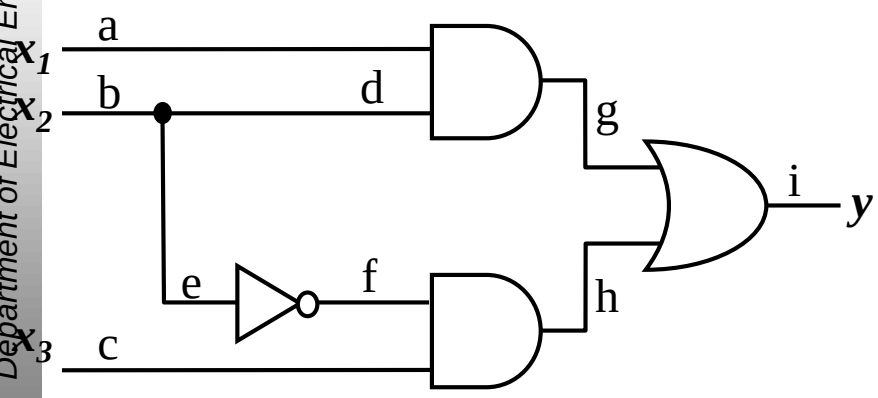
  100 detects d SA1
  - Detect total of 10 faults
  - 001 and 110 detect remaining 8 faults



$x_1x_2x_3$	000	001	010	011	100	101	110	111
y	0	1	0	0	0	1	1	1
a SA0	0	1	0	0	0	1	0	0
a SA1	0	1	1	1	0	1	1	1
b SA0	0	1	0	1	0	1	0	1
b SA1	0	0	0	0	1	1	1	1
c SA0	0	0	0	0	0	0	1	1
c SA1	1	1	0	0	1	1	1	1
d SA0	0	1	0	0	0	1	0	0
d SA1	0	1	0	0	1	1	1	1
e SA0	0	1	0	1	0	1	1	1
e SA1	0	0	0	0	0	0	1	1
f SA0	0	0	0	0	0	0	1	1
f SA1	0	1	0	1	0	1	1	1
g SA0	0	1	0	0	0	1	0	0
g SA1	1	1	1	1	1	1	1	1
h SA0	0	0	0	0	0	0	1	1
h SA1	1	1	1	1	1	1	1	1
i SA0	0	0	0	0	0	0	0	0
i SA1	1	1	1	1	1	1	1	1

Stuck-at Faults

- 4 sets of equivalent faults
- # collapsed faults = $2 \times (P_O+F_O) + G_I-N_I$
  - P_O = # primary outputs
  - F_O = # fanout stems
  - G_I = # gate inputs
  - N_I = # inverters

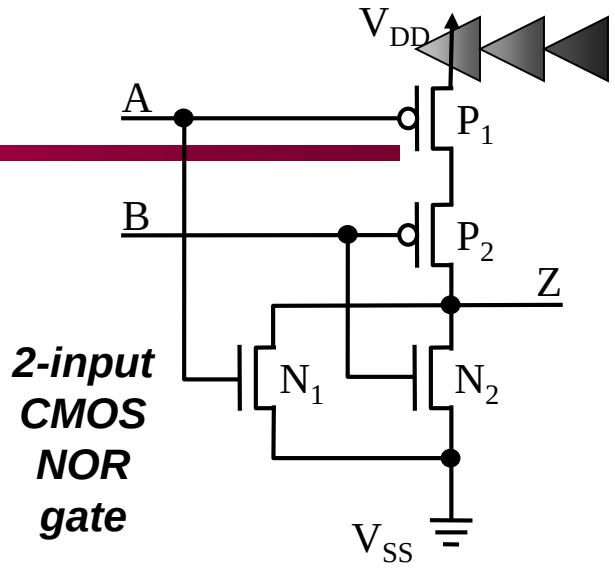


$x_1x_2x_3$	000	001	010	011	100	101	110	111
y	0	1	0	0	0	1	1	1
a SA0	0	1	0	0	0	1	0	0
a SA1	0	1	1	1	0	1	1	1
b SA0	0	1	0	1	0	1	0	1
b SA1	0	0	0	0	1	1	1	1
c SA0	0	0	0	0	0	0	1	1
c SA1	1	1	0	0	1	1	1	1
d SA0	0	1	0	0	0	1	0	0
d SA1	0	1	0	0	1	1	1	1
e SA0	0	1	0	1	0	1	1	1
e SA1	0	0	0	0	0	0	1	1
f SA0	0	0	0	0	0	0	1	1
f SA1	0	1	0	1	0	1	1	1
g SA0	0	1	0	0	0	1	0	0
g SA1	1	1	1	1	1	1	1	1
h SA0	0	0	0	0	0	0	1	1
h SA1	1	1	1	1	1	1	1	1
i SA0	0	0	0	0	0	0	0	0
i SA1	1	1	1	1	1	1	1	1

Transistor Faults

- Any transistor can be
  - Stuck-short
    - Also known as stuck-on
  - Stuck-open
    - Also known as stuck-open

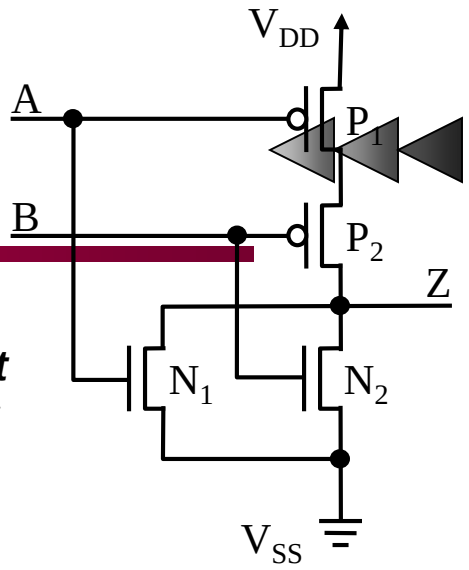
  # fault types: $k = 2$
- Example circuit
  - # fault sites: $n = 4$
  - # single faults = $2 \times 4 = 8$



2-input CMOS NOR gate

AB	00	01	10	11
Z	1	0	0	0
N_1 stuck-open	1	0	last Z	0
N_1 stuck-short	I_{DDQ}	0	0	0
N_2 stuck-open	1	last Z	0	0
N_2 stuck-short	I_{DDQ}	0	0	0
P_1 stuck-open	last Z	0	0	0
P_1 stuck-short	1	0	I_{DDQ}	0
P_2 stuck-open	last Z	0	0	0
P_2 stuck-short	1	I_{DDQ}	0	0

Transistor Faults

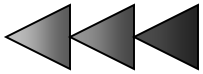


2-input CMOS NOR gate

- Stuck-short faults cause conducting path from V_{DD} to V_{SS}
  - Can be detect by monitoring steady-state power supply current I_{DDQ}
- Stuck-open faults cause output node to store last voltage level
  - Requires sequence of 2 vectors for detection
    - 00 →10 detects N_1 stuck-open

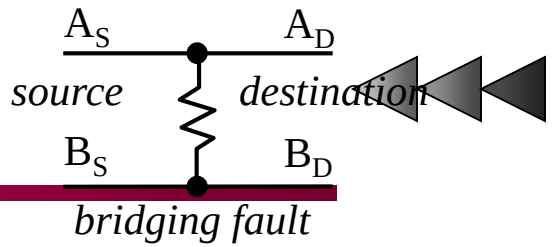
AB	00	01	10	11
Z	1	0	0	0
N_1 stuck-open	1	0	last Z	0
N_1 stuck-short	I_{DDQ}	0	0	0
N_2 stuck-open	1	last Z	0	0
N_2 stuck-short	I_{DDQ}	0	0	0
P_1 stuck-open	last Z	0	0	0
P_1 stuck-short	1	0	I_{DDQ}	0
P_2 stuck-open	last Z	0	0	0
P_2 stuck-short	1	I_{DDQ}	0	0

Shorts and Opens



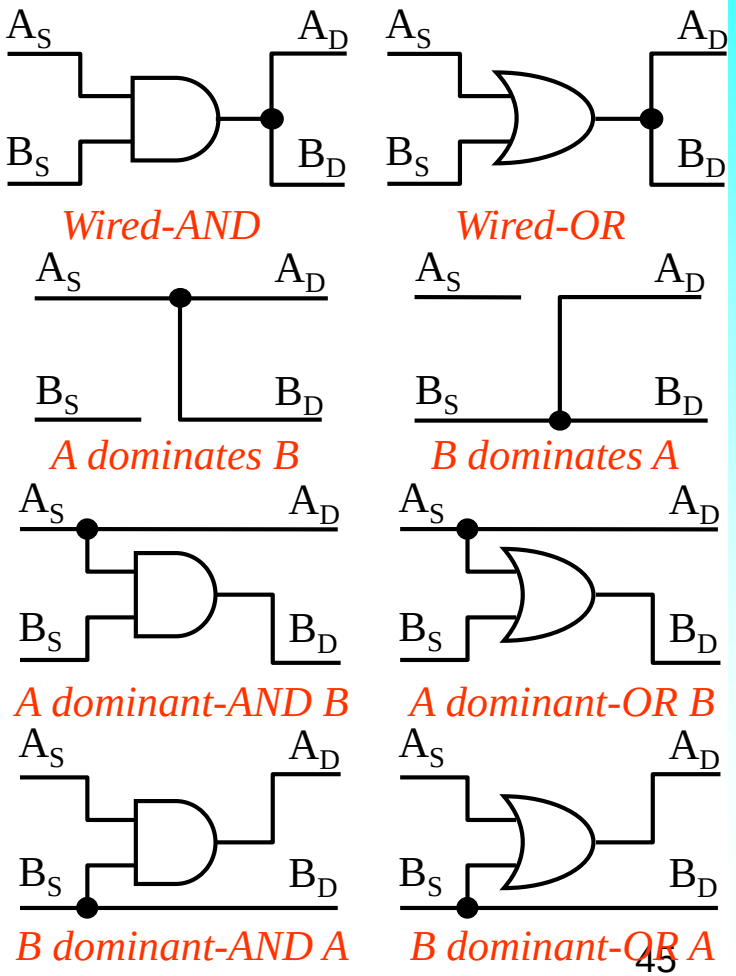
- Wires can be
  - Open
    - Opens in wires interconnecting transistors to form gates behave like transistor stuck-open faults
    - Opens in wires interconnecting gates to form circuit behave like stuck-at faults
    - Opens are detected by vectors detecting transistor and stuck-at faults
  - Short to an adjacent wire
    - Also known as a bridging fault

Bridging Faults



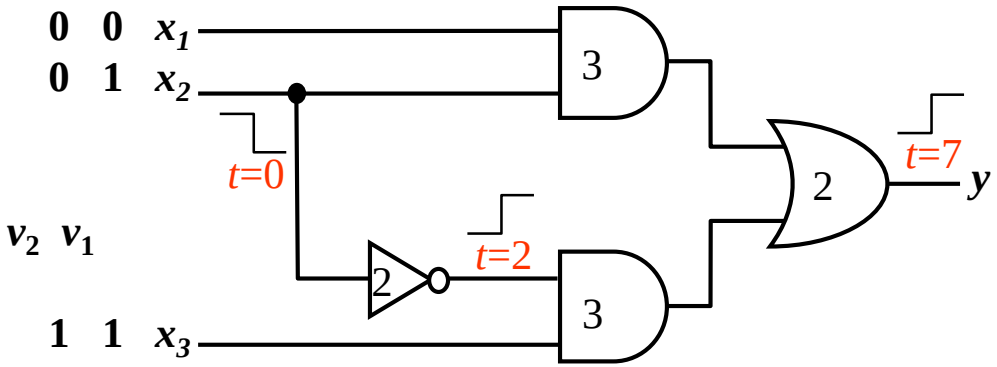
- Three different models
  - Wired-AND/OR
  - Dominant
  - Dominant-AND/OR
- Detectable by I_{DDQ} testing

A_S B_S	0 0	0 1	1 0	1 1
A_D B_D	0 0	0 1	1 0	1 1
Wired-AND	0 0	0 0	0 0	1 1
Wired-OR	0 0	1 1	1 1	1 1
A dominates B	0 0	0 0	1 1	1 1
B dominates A	0 0	1 1	0 0	1 1
A dominant-AND B	0 0	0 0	1 0	1 1
B dominant-AND A	0 0	0 1	0 0	1 1
A dominant-OR B	0 0	0 1	1 1	1 1
B dominant-OR A	0 0	1 1	1 0	1 1

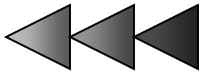


Delay Faults and Crosstalk

- Path-delay fault model considers cumulative propagation delay through CUT
  - 2 test vectors create transition along path
  - Faulty circuit has excessive delay
- Delays and glitches can be caused by crosstalk between interconnect
  - due to inductance and capacitive coupling



Pattern Sensitivity and Coupling Faults



- Common in high density RAMs
- Pattern sensitivity fault
  - Contents of memory cell is affected by contents of neighboring cells
- Coupling fault
  - Transition in contents of one memory cell causes change in contents of another cell

Pattern Sensitivity and Coupling Faults

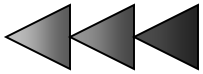
- Common in memory cells of high density RAMs
- Pattern sensitivity fault
  - Contents of cell affected by contents of neighboring cells
- Coupling fault
  - Transition in one cell causes change in another cell
- Detected with specific memory test algorithms
  - Background Data Sequence (BDS) used for word-oriented memories

Notation:

w0 = write 0 (or all 0's)
r1 = read 1 (or all 1's)
↑= address up
↓= address down
↕ = address either way

Test Algorithm	March Test Sequence
March LR w/o BDS	↕(w0); ↓(r0, w1); ↑(r1, w0, r0, r0, w1); ↑(r1, w0); ↑(r0, w1, r1, r1, w0); ↑(r0)
March LR with BDS	↕(w00); ↓(r00, w11); ↑(r11, w00, r00, r00, w11); ↑(r11, w00); ↑(r00, w11, r11, r11, w00); ↑(r00, w01, w10, r10); ↑(r10, w01, r01); ↑(r01)

Analog Fault Models

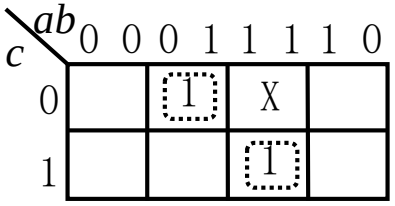


- Catastrophic faults
  - Shorts and opens
- Parametric faults
  - Parametric variations in passive and active components cause components to be out of tolerance range
- Active components can sustain defects that affect DC and/or AC operation

Levels of Abstraction

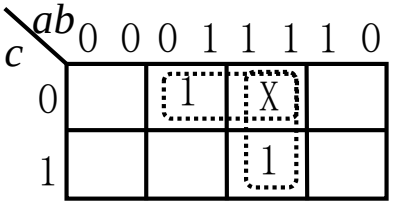
- High levels have few implementation details needed for effective test generation
  - Fault models based on gate & physical levels
- Example: two circuits for same specification
  - Ckt B test vectors do not detect 4 faults in Ckt A

$f(a,b,c)=\Sigma_m(1,7)+d(3) = a\bar{b}\bar{c} + abc + Xab\bar{c}$



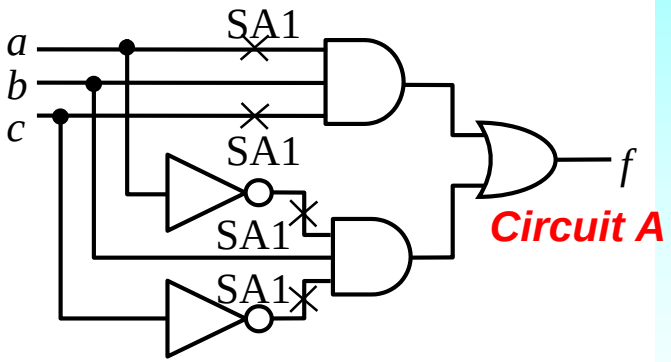
$f = abc + \bar{a}b\bar{c}$

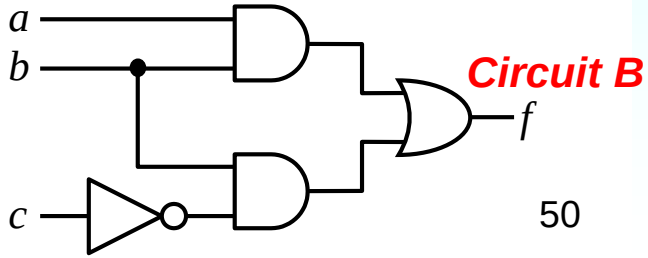
Test Vectors
{111,110,101,011,010,000}



$f = ab + b\bar{c}$

Test Vectors
{111,101,010,000}

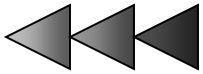




Overview of VLSI Test Technology

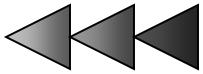
- Automatic Test Equipment (ATE) consists of
  - Computer – for central control and flexible test & measurement for different products
  - Pin electronics & fixtures – to apply test patterns to pins & sample responses
  - Test program – controls timing of test patterns & compares response to known good responses

Overview of VLSI Test Technology



- Automatic Test Pattern Generation (ATPG)
  - Algorithms generating sequence of test vectors for a given circuit based on specific fault models
- Fault simulation
  - Emulates fault models in CUT and applies test vectors to determine fault coverage
  - Simulation time (significant due to large number of faults to emulate) can be reduced by
    - Parallel, deductive, and concurrent fault simulation

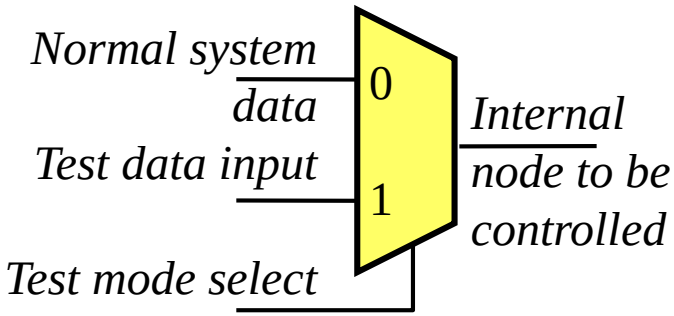
Overview of VLSI Test Technology



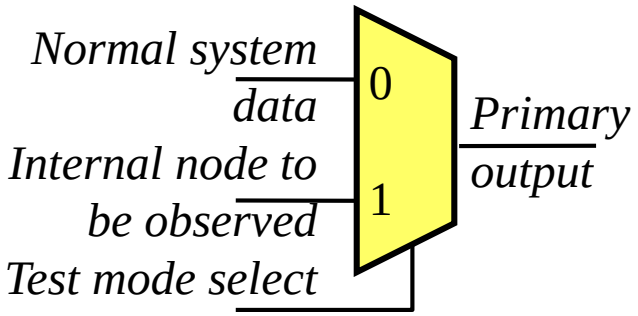
- Design for Testability (DFT)
  - Generally incorporated in design
  - Goal: improve controllability and/or observability of internal nodes of a chip or PCB
- Three basic approaches
  - Ad-hoc techniques
  - Scan design
    - Boundary Scan
  - Built-In Self-Test (BIST)

Design of Testability

- Ad-hoc DFT techniques
  - Add internal test points (usually multiplexers) for
    - Controllability
    - Observability
  - Added on a case-by-case basis
    - Primarily targets “hard to test” portions of chip



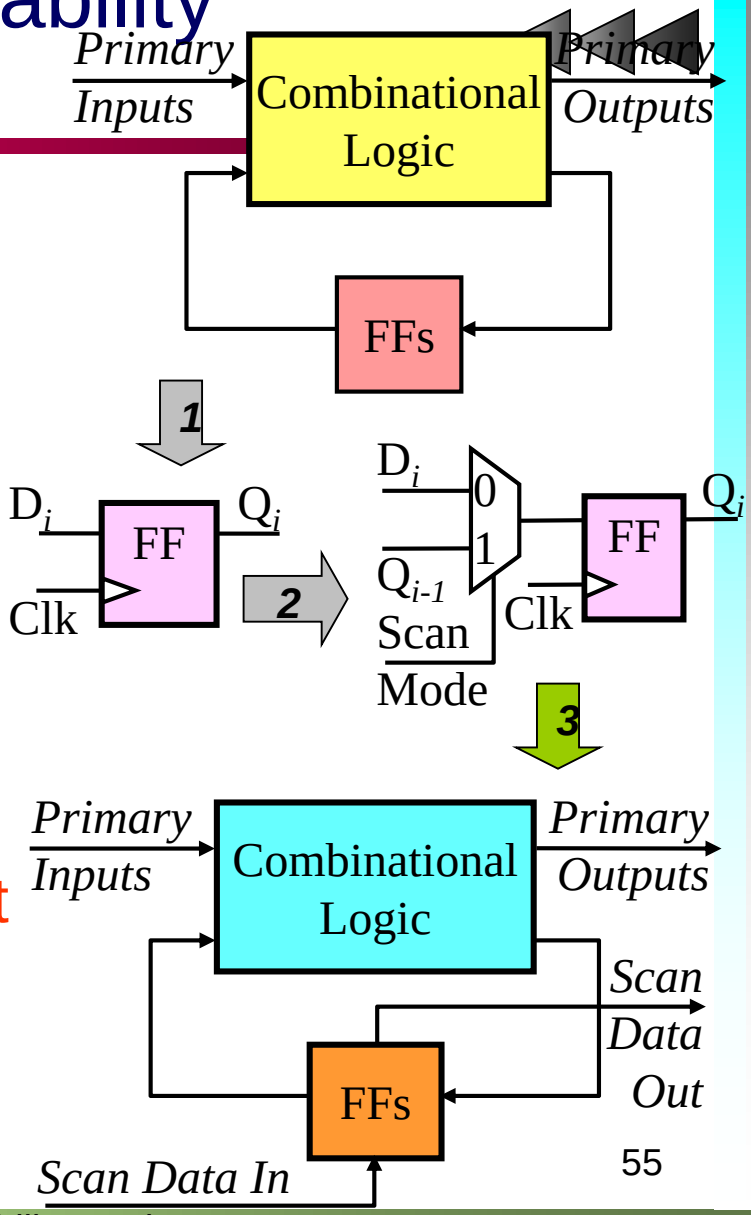
controllability test point



observability test point

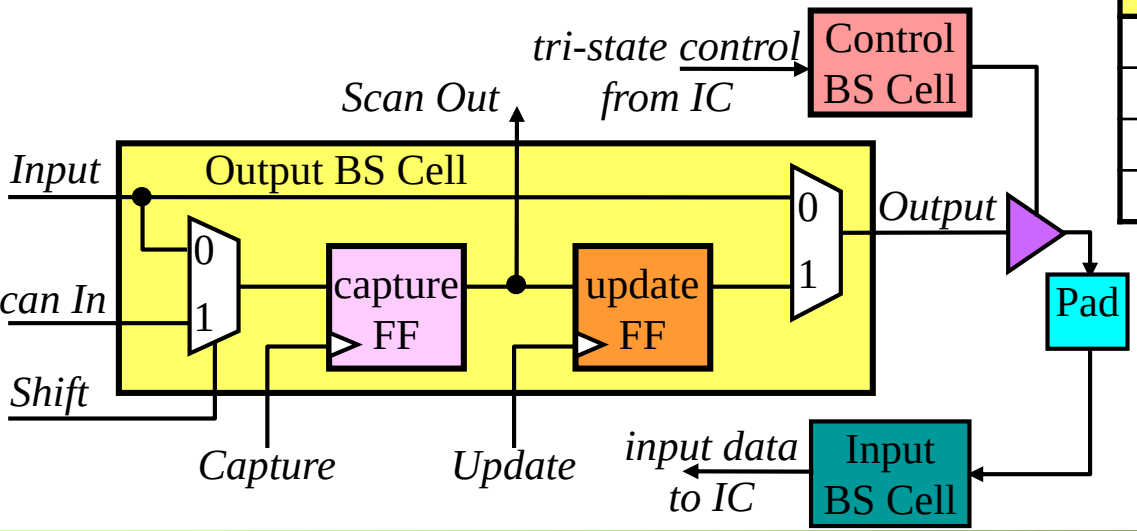
Design for Testability

- Scan design
  - Transforms flip-flops of chip into a shift register
  - Scan mode facilitates
    - Shifting in test vectors
    - Shifting out responses
- Good CAD tool support
  - Transforming flip-flops to shift register
  - ATPG



Design for Testability

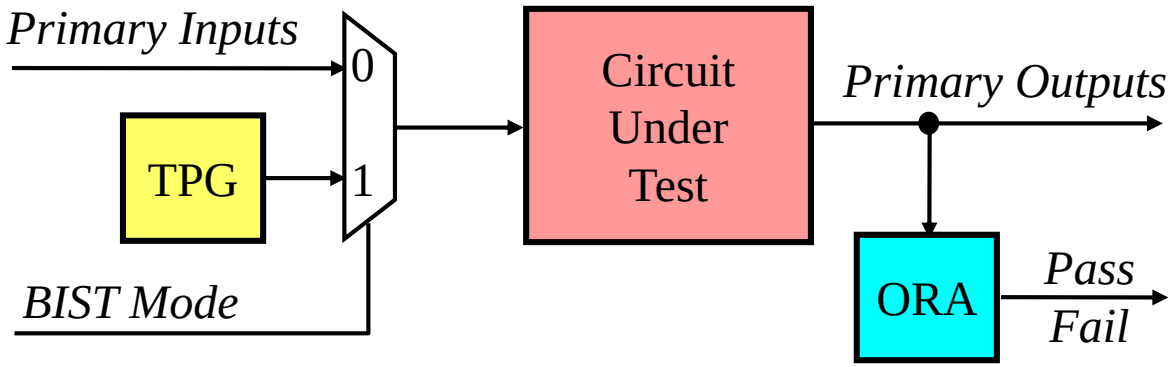
- Boundary Scan – scan design applied to I/O buffers of chip
  - Used for testing interconnect on PCB
    - Provides access to internal DFT capabilities
  - IEEE standard 4-wire Test Access Port (TAP)



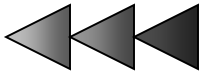
TAP pin	I/O	Function
TCK	input	Test clock
TMS	input	Test Mode Select
TDI	input	Test Data In
TDO	output	Test Data Out

Design for Testability

- Built-In Self-Test (BIST)
  - Incorporates test pattern generator (TPG) and output response analyzer (ORA) internal to design
    - Chip can test itself
  - Can be used at all levels of testing
    - Device → PCB → system → field operation



Concluding Remarks



- Many new testing challenges presented by
  - Increasing size and complexity of VLSI devices
  - Decreasing feature size
- This chapter presented introduction to VLSI testing
- Remaining chapters present more details as well as solutions to these challenges